

Homework # 2

Due Fri. March 20, 2026

1. Iteration

In mathematics, the Fibonacci numbers form a sequence, the *Fibonacci sequence*, in which each number is the sum of the two preceding ones. The sequence commonly starts from 0 and 1, although some authors start the sequence from 1 and 1 or sometimes (as did Fibonacci) from 1 and 2. Starting from 0 and 1, the first few values in the sequence are:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144.

The Fibonacci numbers were first described in Indian mathematics, as early as 200 BC in work by Pingala on enumerating possible patterns of Sanskrit poetry formed from syllables of two lengths. They are named after the Italian mathematician Leonardo of Pisa, later known as Fibonacci, who introduced the sequence to Western European mathematics in his 1202 book *Liber Abaci*.

Fibonacci numbers appear unexpectedly often in mathematics, so much so that there is an entire journal dedicated to their study, the Fibonacci Quarterly. Applications of Fibonacci numbers include computer algorithms such as the Fibonacci search technique and the Fibonacci heap data structure, and graphs called Fibonacci cubes used for interconnecting parallel and distributed systems. They also appear in biological settings, such as branching in trees, the arrangement of leaves on a stem, the fruit sprouts of a pineapple, the flowering of an artichoke, an uncurling fern, and the arrangement of a pine cone's bracts.

Fibonacci numbers are also strongly related to the golden ratio: Binet's formula expresses the n th Fibonacci number in terms of n and the golden ratio, and implies that the ratio of two consecutive Fibonacci numbers tends to the golden ratio as n increases.

Design a set of molecular reactions that implements the Fibonacci procedure for exactly 12 steps. Demonstrate that your code works for starting values 0, 1

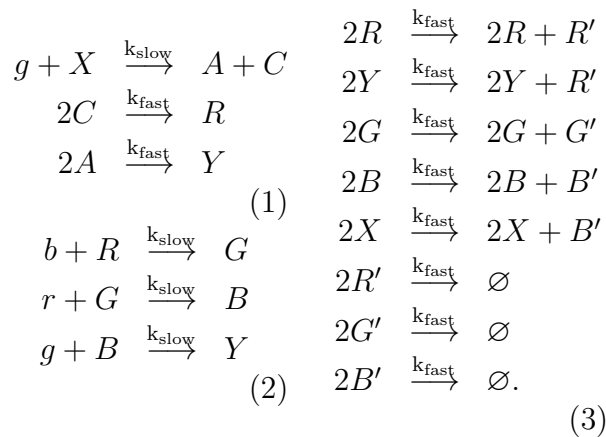
(so it produce 144, as in the example above). Also demonstrate it for starting values 3, 7.

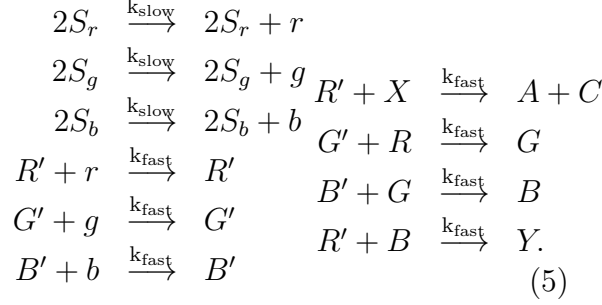
2. Sequential Computation

A sequential system computes output values that are a function of the current input values as well as prior input values. Here “current” and “previous” refer to successive signal values that are supplied by some external source. Our system consumes the input molecular types, and so resets the input signal to zero. Our system accepts new input values only after the current output value is cleared, i.e., after some external source consumes all of the output molecular type.

We illustrate our design methodology with a detailed example: a finite impulse response (FIR) filter. An FIR filter is shown in Figure 1a. This system computes a *moving average*: given a time-varying input signal X , the output Y is a smoother version of it. More precisely, the output is one-half the current input value plus one-half the previous value.

Our implementation consists of the following set of reactions. We present the reactions in their entirety and then provide the rationale for the design.





(4)

The molecular types corresponding to *signals* are X , A , C , R , G , B and Y . These are labeled in Figure 1b. To elucidate the design, we color-code some of these types into three categories: Y and R in red; G in green; and X and B in blue.

In the group of reactions (1), the concentration of X is transferred to A and to C , a *fanout* operation. The concentrations of A and C are both reduced to half – *scalar multiplication* operations. The concentration of A is transferred to the output Y and the concentration of C is transferred to R . (The transfer to R is the first phase of a *delay* operation. We discuss this operation below.) Once the signal has moved through the delay operation, the concentration of B is transferred to the output Y . Since this concentration is combined with the concentration of Y produced from A , this is an *addition* operation.

The group of reactions (2) implements the delay operation. The concentration of R is transferred to G and then to B . Transfers between two color categories are enabled by the absence of the third category: red goes to green in the absence of blue; green goes to blue in the absence of red; and blue goes to red in the absence of green. The reactions are enabled by molecular types r , g , and b that we call *absence indicators*. (We discuss these types below.) The absence indicators ensure that the delay element takes a new value only when it has finished processing the previous value.

In the group of reactions (3), molecules of types R' , G' , and B' are generated from the signal types that we color-code red, green, and blue respectively. The concentrations of the signal types remain unchanged. (These reactions appear to violate conservation of mass. In fact, when mapped to DNA reactions, there are external “fuel” types.) Meanwhile, R' , G' , and B' are consumed by external sinks, denoted by $\emptyset$. (When mapped to DNA, these reactions include

“waste” types.) Here, all reactions are expressly designed to have two reactants; this permits us to map the reactions to DNA strand displacement reactions effectively. This generation/consumption process ensures that equilibria of the concentrations of R' , G' , and B' reflect the total concentrations of red, green, and blue color-coded types, respectively. Accordingly, we call R' , G' , and B' *color concentration indicators*. They serve to speed up signal transfers between color categories.

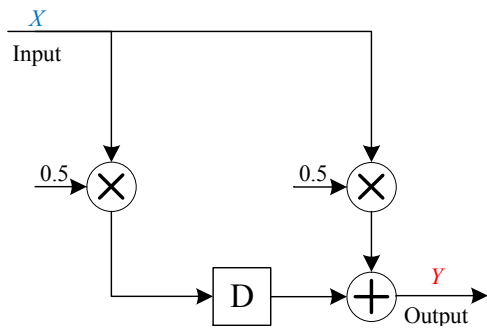
In the group of reactions (4), molecules of the absence indicator types r , g , and b are generated from external sources S_r , S_g , and S_b . At the same time, they are consumed when R' , G' , and B' are present, respectively. Therefore, the absence indicators only persist in the absence of the corresponding signals: r in the absence of red types; g in the absence of green types; and b in the absence of blue types. They only persist in the absence of these types because otherwise “fast” reactions consume them quickly.

Finally, the group of reactions (5) provides positive feedback kinetics. These reactions effectively speed up transfers between color categories as molecules in one category are “pulled” to the next by color concentration indicators.

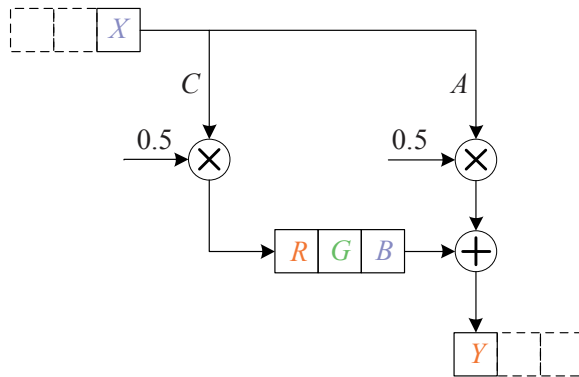
Note that the concentration of the input X is sampled in the green-to-blue phase. We assume that an external source supplies the input. The output Y is produced in the blue-to-red phase. We assume that an external sink consumes these molecules.

Problem. Implement the biquad filter shown in Figure 2a. Provide chemical reactions and simulate the operation of the filter for 5 RGB cycles. Use input values of $X = 100, 5, 500, 20, 250$ for each cycle.

I suggest that you run a simulator like Aleae or Marcea 10 times: first with the input values; record the output value of Y ; then set the output value of Y to zero to simulate “sampling” the output; then set the next input value for X and use all the previous output values of the other variables as new inputs; and run the next cycle.

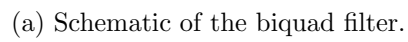


(a) Schematic of the two-tap moving average filter.



(b) The filter in a three-phase configuration.

Figure 1: A two-tap moving average filter.



6